CodeCoR: An LLM-Based Self-Reflective Multi-Agent Framework for Code Generation

Ruwei Pan
Chongqing University
Chongqing, China
panruwei@stu.cqu.edu.cn

Hongyu Zhang*
Chongqing University
Chongqing, China
hongyujohn@gmail.com

Chao Liu
Chongqing University
Chongqing, China
liu.chao@cqu.edu.cn

arXiv:2501.07811v2 [cs.SE] 17 Jul 2026

Abstract

Code generation aims to produce code that fulfills requirements written in natural language automatically. Large Language Models (LLMs) like ChatGPT have demonstrated promising effectiveness in this area. Nonetheless, the LLMs often fail to ensure the syntactic and semantic correctness of the generated code. Recently, researchers proposed multi-agent frameworks that guide LLMs with different prompts to analyze programming tasks, generate code, and perform testing in a sequential workflow. However, the performance of the workflow is not robust as code generation depends on the performance of each agent. To address this challenge, we propose CodeCoR, a multi-agent framework that prunes intermediate outputs at different stages to reduce error propagation in sequential code generation workflows. Specifically, for a given task description, four agents in CodeCoR generate prompts, code, test cases, and repair advice, respectively. Each agent generates more than one output and prunes away the low-quality ones. The generated code is tested in the local environment: the code that fails to pass the generated test cases is sent to the repair agent, and the coding agent re-generates the code based on repair advice. Finally, the code that passes the highest number of generated test cases is returned to the users. Our experiments on four widely used datasets, HumanEval, HumanEval-ET, MBPP, and MBPP-ET, demonstrate that CodeCoR outperforms existing baselines (e.g., CodeCoT and MapCoder), achieving an average Pass@1 score of 77.13%.

CCS Concepts

• **Software and its engineering → Automatic programming**.

Keywords

Code Generation, Multi-Agent, Self-reflection

ACM Reference Format:
Ruwei Pan, Hongyu Zhang, and Chao Liu. 2026. CodeCoR: An LLM-Based Self-Reflective Multi-Agent Framework for Code Generation. In *Proceedings of The 17th International Conference on Internetware (Internetware '26).* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/XXXXXXX.XXXXXXX

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Internetware '26, Gold Coast, QLD, Australia
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/26/07
https://doi.org/10.1145/XXXXXXX.XXXXXXX

1 Introduction

Code generation aims to automatically produce code that fulfills the requirements expressed in natural language [9, 33, 34, 55]. Successful code generation can significantly enhance the productivity and quality of software development, and remains a pivotal area of research in artificial intelligence, natural language processing, and software engineering [13]. In recent years, Large language models (LLMs) such as ChatGPT have demonstrated promising performance in code generation with advanced language understanding and generation capabilities [46]. This progress sets the stage for exploring new prompting techniques and frameworks to further improve code generation.

The Chain-of-Thought (CoT) prompting method is commonly utilized to help LLMs better understand tasks by clarifying the prompts through step-by-step reasoning [47]. This approach helps an LLM clarify its reasoning process for complex tasks. Building on CoT, researchers have proposed structured prompting techniques. For example, Li et al. [25] developed a Structured Chain-of-Thought (SCoT), which mitigates the limitations of CoT by employing a predefined sequence of semantic steps that closely align with programming paradigms, including control structures such as loops and conditional branches. Jiang et al. [19] observed that CoT often suffers from disorganized and inefficient reasoning and introduced a self-planning CoT that guides the prompt generation process based on the code's objectives.

Recently, researchers proposed multi-agent frameworks that guide LLMs through sequential steps to analyze programming tasks, generate code, and repair bugs. This sequential workflow is widely adopted in code generation, including ReAct [51], Reflexion [42], Self-Edit [52], Self-Planning [19], CodeChain [22] and so on. In the domain of code generation, CodeCoT [16] and MapCoder [18] are two representative multi-agent frameworks. CodeCoT [16] involves three agents for task understanding, test case generation, and code generation. The generated code is required to pass the generated test cases; otherwise, the agent will generate new code. Meanwhile, MapCoder [18] is a state-of-the-art multi-agent framework that generates similar step-by-step solution descriptions for a given programming task, then generates the corresponding code, and finally repairs bugs with the test cases provided in testing data. These works demonstrate the effectiveness of sequential multi-agent frameworks in code generation.

However, we observed that the performance of existing multi-agent frameworks is not robust, as the performance of each agent can significantly affect the quality of the final output. As illustrated in Figure 1, in a sequential workflow, once the prompt agent misunderstands the intent of the task description, the misunderstanding will propagate to the follow-up coding agent and test agent. The

error is amplified and the generation effort is wasted. In Figure 1, the task requires a function that removes duplicate elements from a list while preserving the order of their first occurrence. However, the prompt agent focuses on the deduplication requirement but overlooks the order-preservation constraint. The coding agent then generates code based on `set()`, which removes duplicates but does not guarantee the original order. Meanwhile, the test agent produces test cases that check whether duplicates are removed, but fail to verify whether the first-occurrence order is preserved. As a result, the generated code can pass the generated tests but still violate the original requirement. This example highlights the need for a more robust approach that can prune low-quality outputs during the code generation process rather than only at the end.

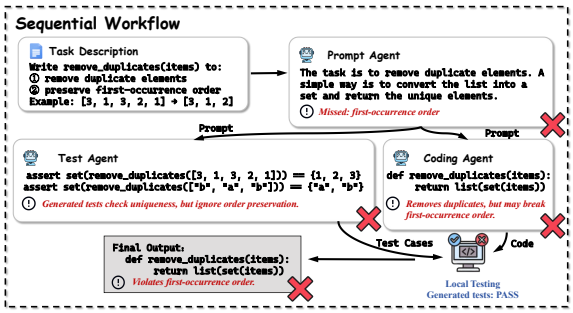

Figure 1: An example of misunderstanding in sequential workflow.

In this study, we introduce CodeCoR ("**Code Co**llaboration and **R**eflection"), a self-reflective multi-agent framework for code generation. In CodeCoR, self-reflection is realized by checking and pruning intermediate outputs before they are used by downstream agents, thereby reducing error propagation in sequential multi-agent workflows. Specifically, CodeCoR consists of four LLM-based agents: ***1) prompt agent***, which generates prompts with the CoT technique [47] for task understanding; ***2) coding agent***, which generates code for a given programming description; ***3) test agent***, which generates test cases according to the task; and ***4) repair agent***, which generates repair advice for the code. The generated code is tested in the local environment; code that fails to pass the generated test cases is sent to the repair agent, and the coding agent regenerates code based on repair advice. Finally, the code that passes the highest number of generated test cases is returned to the users. Unlike existing works, CodeCoR checks intermediate outputs before they are used by downstream agents. Specifically, it generates multiple candidates and prunes low-quality prompts, test cases, code snippets, and repair advice at their corresponding stages.

We evaluated the proposed framework, CodeCoR, on widely used datasets [6, 11] including HumanEval, HumanEval-ET, MBPP, and MBPP-ET. Experimental results show that CodeCoR outperforms representative prompting and multi-agent baselines, including CodeCoT and MapCoder. In particular, CodeCoR achieves an average Pass@1 score of 77.13%, compared with 72.8% achieved by MapCoder. Moreover, we conducted ablation studies to confirm the necessity of each major component of CodeCoR.

In summary, our major contributions include:

- We propose CodeCoR, a self-reflective multi-agent framework involving four collaborative agents (i.e., Prompt Agent, Coding Agent, Test Agent, and Repair Agent) for code generation. The self-reflection in CodeCoR is realized by checking and pruning low-quality or misaligned intermediate outputs before they are passed to downstream agents, reducing error propagation in sequential multi-agent workflows.
- We demonstrate the effectiveness of CodeCoR through extensive experiments on multiple datasets. The experimental results show that CodeCoR outperforms representative prompting and multi-agent baselines, and ablation studies further validate the contribution of its major components.

2 Related Work

2.1 Automatic Code Generation

It has gained increasing attention in software engineering, with early efforts leveraging large language models (LLMs) such as GPT-2 [37] and GPT-3 [2] to handle substantial data demands. Subsequent work extended these models to pass given test cases by filtering outputs [7] and incorporating rule-based sample ranking or complex test generation [23] [3].

Recent studies increasingly focus on improving code quality through iterative self-revision based on feedback. Self-Edit [52] employs test case results for self-revision, while Self-Correct [48] and CodeRL [21] use secondary models to refine output correctness. The technique proposed by Olausson et al. [31] enhances code revision through natural language reflections. Rozière et al. [41] and Huang et al. [17] highlight the importance of large-scale pre-training and robust testing to improve code quality. Jiang et al. [19] propose Self-Planning, which enhances code generation by structuring prompt creation for semantic correctness, but faces challenges with complex tasks, where balancing syntax and semantics remains difficult. This limitation led to the development of the CodeCoR framework.

2.2 Multi-Agent Coding Models

Early multi-agent frameworks focused on basic collaboration strategies [43]. Dong et al. [10] introduced the Self-Collaboration Framework, where LLMs adopt roles like analyst, coder, and tester, using role-specific instructions to enhance task performance. Experiments demonstrate that this self-collaborative approach significantly improves code quality and efficiency. Moreover, SWE-Agent [49] and CodeAgent [53] improve code correctness through external tools and environment-level interactions.

Recent advancements in multi-agent frameworks include MetaGPT [15], which uses human-like SOPs and an executive feedback mechanism to debug and execute code, improving performance on benchmarks like HumanEval. ChatDev [36] enhances collaboration by allowing agents to seek clarifications, reducing incorrect responses. Additionally, the CodeCoT framework [16] improves code reliability by integrating self-examination to address syntax errors and ensure both semantic and syntactical correctness. Despite advancements, multi-agent frameworks like MapCoder [18], which uses retrieval, planning, coding, and debugging agents, face robustness challenges due to their dependency on each agent's performance in

a sequential workflow. CodeCoR addresses this limitation by checking and pruning intermediate outputs before they are propagated to downstream agents.

3 Methodology

3.1 Overview

Figure 2 depicts the CodeCoR process, which includes four LLM-based agents: **Prompt Agent, Test Agent, Coding Agent, and Repair Agent**, whose prompts are illustrated in Figure 3. In the Figure, each agent provides an example of its output, when the code task is to write a function that returns the decimal part of a floating-point number. Each agent is assigned a specific task within the generation phases: **Prompt Generation, Test Case Generation, Code Generation, and Repair**.

Phase-I: Prompt Generation. The process begins with Phase-I: Prompt Generation. Initially, the task description is provided to the Prompt Agent, which generates a series of CoT prompts and stores them in the CoT Pool. The Prompt Agent focuses on generating high-quality CoT prompts. A pruning method (shown in Section 3.3) is employed to reserve the CoT prompts that decompose the task well, offering detailed step-by-step instructions that facilitate the subsequent generation of code and test cases.

Phase-II: Test Case Generation. The Test Agent generates a variety of test cases guided by the selected CoT prompts, which are subsequently stored in the Test Case Pool. To prune low-quality or redundant test cases, the Test Agent uses a pruning method to select only those that effectively assess the executability of the code. This ensures that the test cases are robust and capable of accurately verifying whether the generated code meets the expected requirements.

Phase-III: Code Generation. Once the CoT prompts are selected, they are passed to the Coding Agent, which generates a variety of code snippets, thereby forming the Code Snippet Pool. The Coding Agent also employs a pruning method to identify and select promising code snippets that demonstrate potential for correctness and efficiency. This approach significantly enhances the quality and accuracy of the final code, reducing the need for further revisions and repairs.

Phase-IV: Result Checking. Upon entering Phase-IV, the code generated by the Coding Agent is evaluated against the test cases provided by the Test Agent, and errors are identified by executing the code in a local environment. If all test cases pass, the code is stored in the Ranked Code Set. If the code fails some generated test cases, it is added to the Failed Code Set for repair. The repair process is controlled by the maximum repair round $R_{\max}$. If a code snippet still fails after reaching $R_{\max}$, it is added to the Ranked Code Set.

Phase-V: Repair. Otherwise, the process proceeds to Phase-V. In this phase, the Repair Agent generates repair advice based on the failed code snippets, and a pruning method is employed to eliminate low-quality advice. The advice, along with the erroneous code, is forwarded to the Coding Agent for correction, generating repaired code snippets. The repaired code snippets undergo further pruning and testing. Failed candidates are placed into a new Failed Code Set for the next repair round. The repair process stops when the Failed Code Set becomes empty or when the repair round reaches $R_{\max}$. Finally, the highest-ranked code is returned to the user.

3.2 Pruning Methods

Traditional multi-agent frameworks guide LLMs with different prompts to analyze programming tasks, generate code, test with generated test cases, and repair tested bugs in a sequential workflow. However, the workflow is not robust as the code generation depends on the performance of each agent. Consequently, CodeCoR realizes self-reflection through pruning methods that check intermediate outputs at different stages before they affect downstream agents, thereby reducing error propagation across the sequential workflow. Figure 4 illustrates the pruning process of the CodeCoR framework, detailing the interactions and functionalities of its components. Recently, Wang et al. [45] showed that LLMs can be used as auxiliary evaluators for software engineering tasks. In CodeCoR, LLM-based evaluation is used only as a lightweight heuristic filter for intermediate outputs, rather than as the final correctness oracle. Final code correctness is still evaluated using execution-based benchmark tests. Unlike prior works that rely on fixed metrics or test execution alone, CodeCoR introduces **pruning prompts** that explicitly guide LLMs to assess the quality of candidate prompts, test cases, and code snippets, allowing the framework to automatically prune low-quality outputs. By pruning the low-quality outputs, CodeCoR enhances the efficiency of the multi-agent framework. By ranking the results in the Ranked Code Set, CodeCoR outputs the highest-ranked generated code.

Prompt Pruning. After the Prompt Agent generates a series of CoT prompts for the coding task, it evaluates each prompt's clarity, relevance, conciseness, and context. These criteria are used to check whether a prompt is understandable, task-related, concise, and sufficiently informative. An example prompt is shown in Figure 5. Scores for pruning are assigned by the Prompt Agent based on specific evaluation criteria. Each 1 or 0 indicates whether a particular criterion is met or not. The specific criteria are as follows: ❶ **Clarity:** whether the prompt or advice is understandable and free from ambiguous instructions. ❷ **Relevance:** whether it focuses on the current task or the observed failure. ❸ **Conciseness:** whether it avoids unnecessary or repetitive content. ❹ **Context:** whether it provides enough information for the downstream agent to continue generation or repair. We use binary scores because pruning only decides whether a candidate should be passed to the next stage, rather than producing a fine-grained quality ranking. A prompt is retained only when it receives [1, 1, 1, 1]. We use this all-positive rule because failing any criterion may pass ambiguous, irrelevant, redundant, or insufficient information to the next stage.

Test Pruning. After the Test Agent generates test cases, it uses another prompt to classify them. Empty input, incomplete format test cases, or invalid test cases are pruned. Specifically, empty inputs are cases where no data is provided, incomplete format test cases lack significant components necessary for execution, and invalid test cases are cases without the expected data types or falling outside reasonable ranges.

Code Pruning. The Coding Agent executes the generated code snippets in the local environment. If code contains syntax errors, such as missing semicolons, unmatched parentheses, unclosed

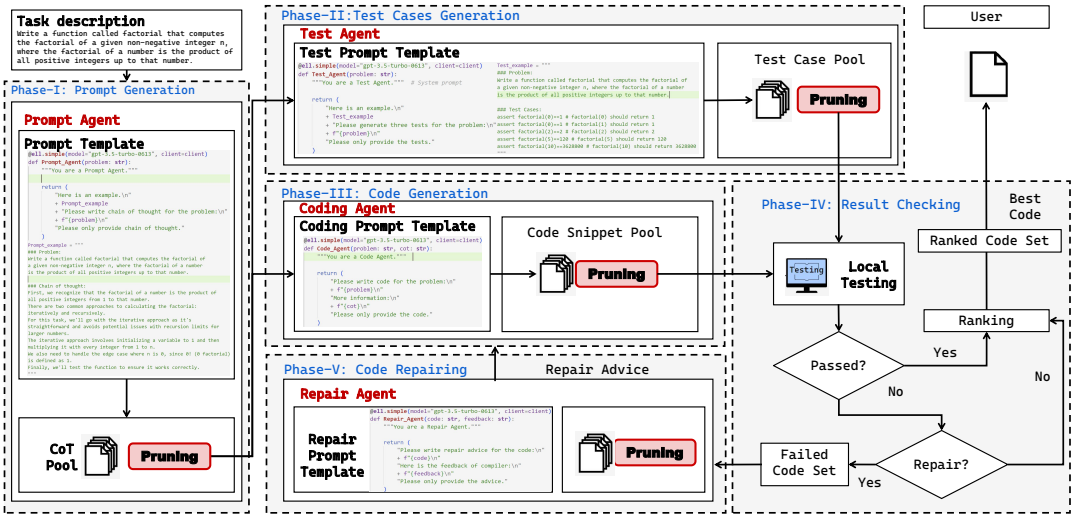

Figure 2: Overview of the CodeCoR framework

strings, or incorrect indentation, it cannot be compiled and will be pruned by the Coding Agent.

Repair Pruning. For each failed code snippet, the Repair Agent provides a single piece of repair advice per repair round. If the repair advice cannot meet clarity, relevance, conciseness, and context requirements (as evaluated by the Repair Agent with the specific prompt in Appendix), the advice is pruned and the failed test cases replace the repair advice. Then, the failed test cases are directly submitted, along with the failed code, to the Coding Agent for repair.

By employing these pruning methods, CodeCoR performs self-reflection through intermediate output checking and reduces the propagation of low-quality outputs across stages.

3.3 Overall Algorithm

The CodeCoR process algorithm (Algorithm 1) generates and repairs code to ensure the final output is accurate. In **Phase-I**, based on the task description T_d, the Prompt Agent generates a set of CoT prompts `CoTPrompts` and prunes unpromising ones. In **Phase-II**, the Test Agent uses the selected prompts to generate test cases `TestCases` and prunes them to ensure that they are robust and executable. In **Phase-III**, the Coding Agent takes `CoTPrompts` as input to generate multiple candidate code snippets `CandidateCodes`. It then applies `prune_code` to retain promising candidates and improve the efficiency of the subsequent phases. For all pruning operations, CodeCoR uses `prune_or_fallback`: if pruning removes all candidates at a stage, the framework keeps the highest-scored candidate according to the corresponding pruning criteria. This prevents the pipeline from stopping due to over-pruning. In **Phase-IV**, each snippet in `CandidateCodes` is tested against `TestCases` using `execute_code`. If the execution `result` is "pass", the snippet is added to the ranked set `RankedSet`. Otherwise, it is added to the failed set `FailedSet` for repair. In **Phase-V**, the Repair Agent generates repair suggestions `RepairSuggestions` via `generate_repair` and prunes them with `prune_repair`. The suggestions are applied to the failed set `FailedSet`, producing updated code snippets `RepairedCodes`. These snippets are tested again, and failed snippets are placed into a new failed set for the next repair round. The process iterates until `FailedSet` becomes empty or the repair round reaches $R_{\max}$. Any remaining failed snippets after the repair

Algorithm 1 CodeCoR Process

Require: Task description T_d, maximum repair rounds $R_{\max}$
Ensure: Final code snippet C_f

$RankedSet \leftarrow [\,]$
$FailedSet \leftarrow [\,]$
Phase-I: Prompt Generation (Prompt Agent)
$CoTPrompts \leftarrow generate_CoT_prompts(T_d)$
$CoTPrompts \leftarrow prune_or_fallback(CoTPrompts, prune_CoT_prompts)$
Phase-II: Test Case Generation (Test Agent)
$TestCases \leftarrow generate_tests(CoTPrompts)$
$TestCases \leftarrow prune_or_fallback(TestCases, prune_tests)$
Phase-III: Code Generation (Coding Agent)
$CandidateCodes \leftarrow generate_code(CoTPrompts)$
$CandidateCodes \leftarrow prune_or_fallback(CandidateCodes, prune_code)$
# Phase-IV: Result Checking
for $codeSnippet \in CandidateCodes$ **do**
  $(result, error) \leftarrow execute_code(codeSnippet, TestCases)$
  if $result ==$ "pass" **then**
    $RankedSet.append(codeSnippet)$
  else
    $FailedSet.append(codeSnippet)$
Phase-V: Repair (Repair Agent)
$repairRound \leftarrow 0$
while $FailedSet \neq \emptyset$ and $repairRound < R_{\max}$ **do**
  $repairRound \leftarrow repairRound + 1$
  $RepairSuggestions \leftarrow generate_repair(FailedSet)$
  $RepairSuggestions \leftarrow prune_or_fallback(RepairSuggestions, prune_repair)$
  $RepairedCodes \leftarrow apply_repair(RepairSuggestions)$
  $NewFailedSet \leftarrow [\,]$
  for $codeSnippet \in RepairedCodes$ **do**
    $(result, error) \leftarrow execute_code(codeSnippet, TestCases)$
    if $result ==$ "pass" **then**
      $RankedSet.append(codeSnippet)$
    else
      $NewFailedSet.append(codeSnippet)$
  $FailedSet \leftarrow NewFailedSet$
for $codeSnippet \in FailedSet$ **do**
  $RankedSet.append(codeSnippet)$
$C_f \leftarrow select_best(RankedSet)$ **return** C_f

Figure 3: Examples of prompts and outputs for four agents.

loop are added to RankedSet for final ranking. Finally, CodeCoR outputs the highest-ranked code snippet from RankedSet using select_best, ensuring that the final output C_f is both accurate and efficient.

4 Evaluation

4.1 Research Questions

This study aims to evaluate CodeCoR by answering the following research questions (RQs):

RQ1. How effective is CodeCoR for function-level code generation? RQ2. How effective are the major components of CodeCoR?

RQ3. What are the cost implications of CodeCoR?

RQ4. How does CodeCoR perform on repository-level code generation?

4.2 Datasets

We evaluate CodeCoR's effectiveness utilizing four widely adopted code generation datasets: HumanEval [6] and MBPP [1], along with their enhanced versions, HumanEval-ET and MBPP-ET [11]. HumanEval and HumanEval-ET are designed to provide diverse programming challenges to assess the model's problem-solving capabilities and adaptability. HumanEval is a hand-written evaluation

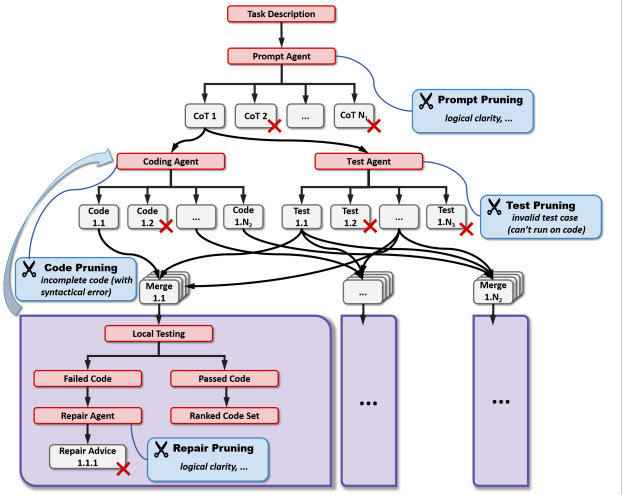

Figure 4: The hierarchical structure of the pruning process

dataset consisting of 164 Python programming problems that assess functional correctness by evaluating language comprehension, reasoning, algorithms, and simple mathematics through unit tests. These problems involve handling specific programming scenarios, necessitating the model to possess comprehension and flexible application skills in Python. Meanwhile, MBPP and MBPP-ET offer around 1,000 crowd-sourced Python programming problems aimed at evaluating the model's proficiency in Python syntax and its ability to handle various coding scenarios.

In addition to these function-level datasets, we include DevEval [28] and RAL-Bench [35] for evaluating CodeCoR in more practical software engineering scenarios. DevEval contains repository-level code generation tasks that require models to complete code within existing projects. RAL-Bench further evaluates application-level repository generation, where the generated repositories are assessed using functional system tests. These benchmarks complement HumanEval and MBPP by involving repository context, project-level dependencies, and execution-based validation beyond standalone programming problems.

4.3 Baselines

We show the efficacy of CodeCoR through comparative analyses with a spectrum of prominent LLMs, encompassing both open-source and proprietary variants (see our repository for details), including Incoder [12], CodeGeeX [54], Starcoder [24], CodeGen-Mono [30], CodeX [3], GPT-3.5-turbo [32], ReAct [51], Reflexion [42], ToT [50], RAP [14], Self-Edit [52], Self-Planning [19], Self-Debugging [4], Self-Collaboration [10], SCoT [27], CodeChain [22], INTERVENOR [44], CodeCoT [16], PaLM Coder [8], Claude-instant-1 [29], GPT-4 [32], MapCoder [18], CodeTree [26], and DebateCoder [5]. These methodologies have been empirically demonstrated to yield substantive enhancements in LLM performance across intricate code generation scenarios. For function-level baselines, we report results from prior studies when they follow the same benchmark setting [16, 18].

4.4 Evaluation Metrics

We employ benchmark-specific metrics to evaluate the correctness and quality of the generated code. First, we utilize the widely adopted **Pass@1** metric, which measures the proportion of generated code snippets that correctly perform the intended task on the first attempt without any modifications [7]. Pass@1 is used as the primary metric for function-level benchmarks, since it directly reflects whether the generated code passes the corresponding test cases. Additionally, to provide a more accurate assessment of the structural and semantic similarity between the generated code and reference implementations, we include **CodeBLEU**, which enhances token-level evaluation with syntactic features through abstract syntax trees (AST) and semantic features through data flow [38]. We use CodeBLEU only as a complementary metric for code similarity, rather than as a substitute for execution-based functional correctness. For RAL-Bench, we follow its original evaluation protocol and report **Fun.**, which measures the average pass rate of functional system tests across generated repositories. Compared with function-level unit tests, Fun. evaluates whether the generated repository satisfies application-level requirements under system-level execution tests.

4.5 Implementation Details

For function-level experiments, we use GPT-3.5-turbo for CodeCoR and all reproduced baselines, with temperature 0.5 and a maximum output length of 1024 tokens. For DevEval and RAL-Bench, we use GPT-4o-mini with temperature 0.5 and maximum output lengths of 4096 tokens, respectively. CodeCoR uses $K_p = 3$, $K_t = 5$, $K_c = 3$, $K_r = 3$, and $R_{\max} = 3$. For controlled comparison, we rerun the main baselines with available released code or prompts under the same model, input, decoding setting, and evaluation protocol. Generated tests are used only for candidate ranking and repair inside CodeCoR. They are not used as the final correctness oracle. All reported Pass@1 results are computed using the official benchmark tests.

4.6 How effective is CodeCoR for function-level code generation? (RQ1)

The evaluation results of CodeCoR and the baselines are shown in Table 1, which show that CodeCoR achieves state-of-the-art performance compared to baseline models on the HumanEval and MBPP datasets. In these evaluations, the GPT-3.5-turbo language model was employed to perform the code generation tasks. In Table 1, GPT-3.5-turbo achieves the highest average Pass@1 score and shows better performance than other Code LLMs. Thus, we adopted GPT-3.5-turbo as the primary backbone model for evaluating our framework, ensuring consistency with prior works and enabling a fairer comparison. In addition, to further assess the generalizability of our approach, we also conducted experiments using more advanced models, including GPT-4-turbo and CodeLlama(34B) shown in Table 6. The evaluation results indicate that CodeCoR outperforms prompting-only models on the HumanEval and MBPP datasets.

On the HumanEval dataset, CodeCoR achieves a Pass@1 score of 86.6%, whereas Self-Planning [19], SCoT [27], and Reflexion score 65.2%, 60.6%, and 68.1%, respectively. On the MBPP dataset,

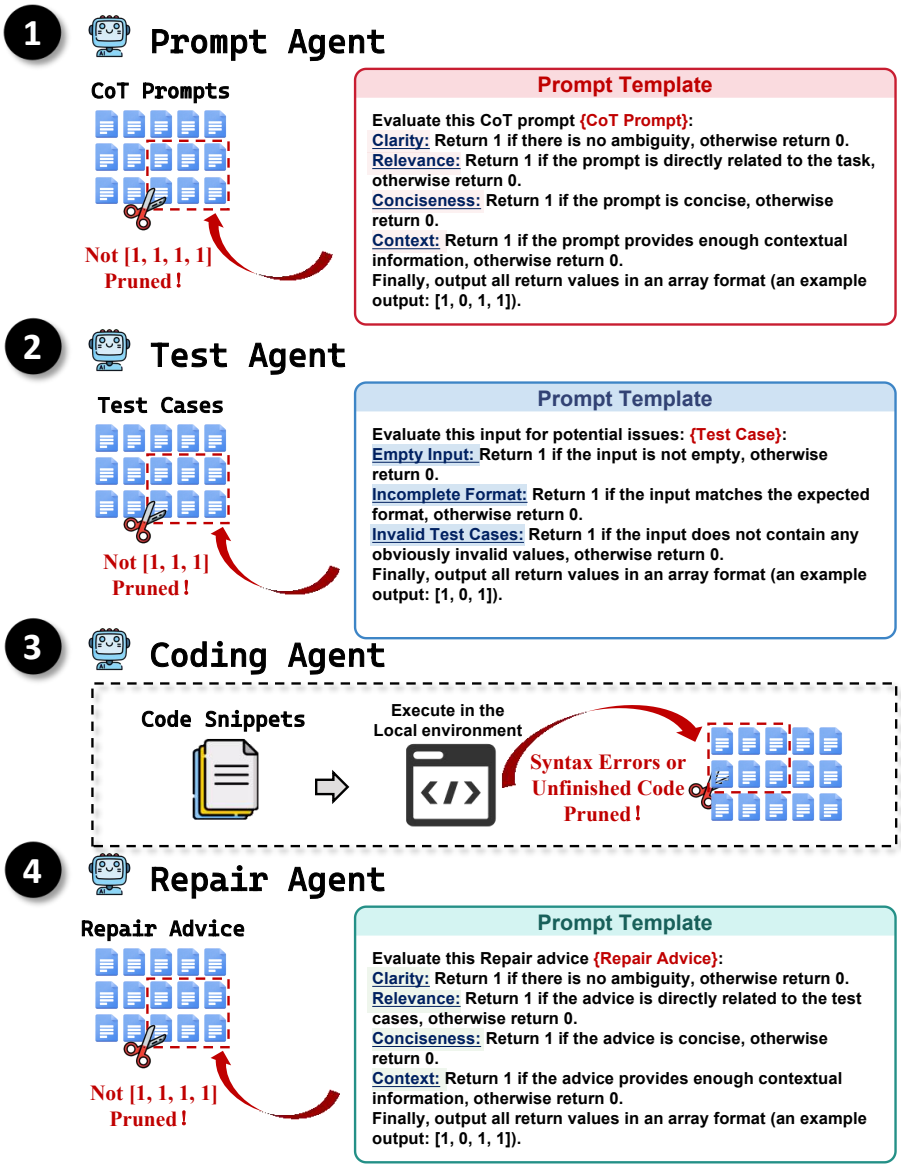

Figure 5: The pruning prompts of agents and the workflow for pruning the low-quality outputs

CodeCoR scores 79.2%, higher than Self-Planning, SCoT, and Reflexion [42], which score 58.6%, 67.0%, and 70.0%, respectively. These results indicate that CodeCoR outperforms existing prompting strategies. When comparing CodeCoR with multi-agent frameworks, CodeCoR also excels. For instance, CodeCoR achieves a Pass@1 score of 86.6% on the HumanEval dataset, whereas MapCoder [18] and CodeCoT [16] achieve 80.5%, and 79.3%, respectively. On the MBPP dataset, CodeCoR scores 79.2%, compared to 78.9% and 67.7% for MapCoder and CodeCoT, respectively. These results demonstrate that CodeCoR enhances code generation effectiveness compared to other multi-agent frameworks.

We also compare CodeCoR with baseline models in terms of CodeBLEU scores on the HumanEval and MBPP datasets. We select four models for comparison. The selection of these four models — SCoT [27], CodeChain [22], Self-Planning [19], and MapCoder [18] — is based on their representativeness and state-of-the-art performance in code generation. Specifically, SCoT, CodeChain, and Self-Planning are chosen for their exceptional semantic capabilities, representing the most advanced semantic baselines, while MapCoder is selected for its state-of-the-art code generation capabilities. Specifically, utilizing the reference solutions and the generated code files in the HumanEval and MBPP datasets, we calculated the CodeBLEU between the generated code and the reference code for each task. For each task, we aligned the standard code with the generated code by task id. Subsequently, we computed CodeBLEU for

Table 1: Comparison of code generation models for HumanEval, HumanEval-ET, MBPP, and MBPP-ET datasets (Pass@1 scores). The best approach is highlighted in bold.

Code LLMs	Human -Eval	Human Eval-ET	MBPP	MBPP -ET	Avg.
Incoder (6.7B)	15.2	11.6	17.6	14.3	14.7
CodeGeeX (1.3B)	18.9	15.2	26.9	20.4	20.4
Claude-instant-1	31.1	28.1	26.9	19.9	26.5
CodeGen-Mono	32.9	25.0	38.6	31.6	31.5
PaLM Coder	43.9	36.6	32.3	27.2	35.0
StarCoder (15.5B)	34.1	25.6	43.6	33.4	34.2
CodeX (175B)	47.0	31.7	58.1	38.8	43.9
GPT-3.5-turbo	**57.3**	**42.7**	**52.2**	**36.8**	**47.3**
Methods	**Human -Eval**	**Human Eval-ET**	**MBPP**	**MBPP -ET**	**Avg.**
Few-Shot	67.7	54.9	65.8	48.3	59.17
ReAct	56.9	49.4	67.0	45.9	54.83
Reflexion	68.1	50.6	70.0	47.4	59.03
ToT	54.4	42.7	65.8	40.8	50.42
RAP	63.1	52.4	71.4	46.7	58.42
Self-Edit	62.2	54.3	56.4	45.9	54.70
Self-Planning	65.2	48.8	58.6	41.5	53.26
Self-Debugging	61.6	45.8	60.1	52.3	54.95
Self-Collaboration	74.4	56.1	68.2	49.5	62.05
INTERVENOR	75.6	54.8	69.8	47.1	61.83
SCoT	60.6	53.4	67.0	51.9	58.22
CodeChain	62.8	54.3	59.1	45.5	55.68
Vanilla CodeCoT	69.5	58.5	67.7	48.6	61.08
CodeCoT	79.3	69.5	67.7	58.1	68.90
MapCoder	80.5	77.4	78.9	54.4	72.80
DebateCoder	67.7	65.2	63.7	37.6	58.55
CodeTree	85.4	79.6	61.2	43.5	67.43
CodeCoR	**86.6**	**80.5**	**79.2**	**65.2**	**77.13**

Table 2: Comparison of models on the HumanEval and MBPP datasets based on CodeBLEU scores, including the mean values across both datasets. The numbers in parentheses indicate the percentage drop compared to CodeCoR.

Method	HumanEval	MBPP	Mean
Self-Planning	50.25 (↓3.64%)	41.62 (↓20.07%)	45.94 (↓11.84%)
SCoT	49.68 (↓4.74%)	34.22 (↓34.29%)	41.95 (↓19.51%)
CodeChain	49.22 (↓5.60%)	45.89 (↓11.88%)	47.56 (↓8.74%)
MapCoder	49.88 (↓4.35%)	48.56 (↓6.74%)	49.22 (↓5.55%)
CodeCoR	**52.15**	**52.08**	**52.12**

each task and aggregated these scores. The final average CodeBLEU scores were derived by averaging the scores across all tasks.

The comparison results are shown in Table 2. On the HumanEval dataset, CodeCoR achieves an average CodeBLEU score of 52.15, outperforming Self-Planning (50.25) and MapCoder (49.88). Although Self-Planning and MapCoder demonstrate comparable results on some individual tasks, CodeCoR's higher average score highlights its superiority in generating semantically accurate and robust code. On the MBPP dataset, CodeCoR again performs exceptionally, achieving an average score of 52.08, closely aligned with its HumanEval performance. When considering the mean CodeBLEU values across both datasets, CodeCoR maintains its leading position, achieving a mean score of 52.12, surpassing all models, including MapCoder (49.22) and CodeChain (47.56). This superior performance reflects that CodeCoR not only produces code that is structurally and semantically closer to the reference solutions but also generalizes effectively across different datasets.

> ***Answer to RQ1:*** CodeCoR achieves higher performance on four widely used datasets over existing LLM-based code generation models; the code generated by CodeCoR shows better CodeBLEU to the ground-truth code.

4.7 How Effective Are the Major Components of CodeCoR? (RQ2)

In this section, we analyze how major components of CodeCoR (as illustrated in Figure 2) affect its effectiveness. We compare the effectiveness of CodeCoR with the following four variants:

❶ **w/o Prompt Agent** This variant of CodeCoR operates without the Prompt Agent. The task description replaces the selected CoT prompts in the CoT Pool and is directly passed to the Test Agent and Coding Agent to guide the generation of test cases and code snippets.

❷ **w/o Test Agent:** This variant is CodeCoR without the Test Agent, which is responsible for validating the syntactic accuracy of the generated code. The goal is to assess whether the absence of the Test Agent leads to an increase in syntax errors. The generated code is executed directly in the local environment. If the code fails, the code and feedback are forwarded to the Repair Agent.

❸ **w/o Repair Agent:** This variant of CodeCoR removes the Repair Agent, which is responsible for correcting errors in the generated code. Its purpose is to verify whether the Repair Agent can enhance the overall quality and reliability of the code by systematically analyzing and fixing these errors. If the generated code cannot pass the test cases generated by the Test Agent, the code and feedback are directly sent to the Coding Agent for repairs.

❹ **w/o Pruning Method:** This variant is CodeCoR without the Pruning Method, which makes CodeCoR not follow the traditional sequential multi-agent framework. It aims to verify whether the Pruning Method can optimize the generation process by improving the efficiency and quality of agent interactions. This variant cannot prune the outputs of the agents and selects the high-quality outputs of the framework.

The evaluation results are shown in Table 3. Each major component affects the performance of CodeCoR. The lack of Prompt Agent results in a reduction in Pass@1 for each dataset: 77.4% for the HumanEval dataset and 70.1% for the HumanEval-ET dataset. This highlights the importance of the Prompt Agent in providing clear context and correct direction for the task, which plays a key role in maintaining the quality and accuracy of the generated code.

Specifically, the absence of the Test Agent results in significant Pass@1 reductions across various datasets: 45.1% on HumanEval, 43.3% on HumanEval-ET, 52.1% on MBPP, and 40.6% on MBPP-ET. This shows the importance of syntactic validation in maintaining the quality and reliability of the generated code. When the Repair Agent is omitted, there are notable declines in Pass@1 scores: 75.6% on HumanEval, 75.6% on HumanEval-ET, 67.7% on MBPP, and 48.6% on MBPP-ET. The Repair Agent plays a critical role in

self-correction, enabling the system to address and rectify errors effectively. Its absence leads to a significant decrease in the overall quality of the code, as the system becomes less capable of correcting errors autonomously. When the Pruning Method is removed, Pass@1 scores drop to 77.4% on HumanEval, 79.2% on HumanEval-ET, 67.7% on MBPP, and 58.1% on MBPP-ET. This module is essential for managing the flow and processing of inputs, ensuring that the generated code remains coherent and of high quality. Its absence can lead to increased noise and errors in the code generation process, resulting in performance degradation.

Table 3: The impact of major components of CodeCoR on HumanEval and MBPP datasets (Pass@1)

Variant	HumanEval	HumanEval-ET	MBPP	MBPP-ET
w/o Prompt Agent	77.4 $^{9.2\downarrow}$	70.1 $^{10.4\downarrow}$	58.4 $^{20.8\downarrow}$	46.4 $^{18.8\downarrow}$
w/o Test Agent	45.1 $^{41.5\downarrow}$	43.3 $^{37.2\downarrow}$	52.1 $^{27.1\downarrow}$	40.6 $^{24.6\downarrow}$
w/o Repair Agent	75.6 $^{11.0\downarrow}$	75.6 $^{4.9\downarrow}$	67.7 $^{11.5\downarrow}$	48.6 $^{16.6\downarrow}$
w/o Pruning Method	77.4 $^{9.2\downarrow}$	79.2 $^{1.3\downarrow}$	67.7 $^{11.5\downarrow}$	58.1 $^{7.1\downarrow}$
CodeCoR	**86.6**	**80.5**	**79.2**	**65.2**

Answer to RQ2: Our CodeCoR framework exhibits better performance than its variants, confirming the effectiveness and necessity of its major components.

5 What are the cost implications of CodeCoR? (RQ3)

The cost implications of multi-agent frameworks are usually higher than those of simpler prompting frameworks. Therefore, we evaluate the system-level cost of CodeCoR and compare it with representative baselines. Table 4 provides an empirical assessment of different code generation frameworks, including MapCoder, CodeTree, DebateCoder, SCoT, Self-Planning, CodeChain, and CodeCoR. The experiment was conducted in the Python environment on the full HumanEval benchmark. We employed `psutil` [39], a Python library, for monitoring costs. We recorded execution time, CPU usage, disk write, and network I/O to evaluate the system-level cost of CodeCoR. The experiments were conducted on a dedicated server to ensure that other services or processes did not influence our measurements.

Table 4: System-level cost comparison on the full HumanEval benchmark.

Method	Run Time(s)	CPU Usage(%)	Disk Write(MB)	Net Send(MB)	Net Receive(MB)
MapCoder	192.59	0.06	2.36	52.25	38.78
CodeTree	101.37	0.05	1.30	2.27	12.57
DebateCoder	255.32	0.05	3.36	17.83	27.43
SCoT	79.69	0.07	1.27	3.56	3.57
Self-Planning	52.48	0.05	0.95	0.74	0.78
CodeChain	119.13	0.05	2.48	19.61	22.20
CodeCoR	91.61	0.06	0.83	22.37	39.59

Table 4 reports the system-level cost comparison on the full HumanEval benchmark. CodeCoR completes the benchmark in 91.61 seconds, which is faster than MapCoder, CodeTree, DebateCoder, and CodeChain, but slower than Self-Planning and SCoT. This result shows that CodeCoR does not achieve the lowest runtime among all methods, but it remains efficient among multi-agent and structured code generation frameworks. For CPU usage and disk writing, CodeCoR remains comparable to or better than most baselines. Its CPU usage is 0.06%, close to the other methods, and its disk write volume is 0.83 MB, which is the lowest among the compared methods. For network traffic, CodeCoR has moderate Net Send but relatively high Net Receive. This is mainly because CodeCoR involves multiple agents and receives more intermediate information during candidate generation, pruning, and repair. Although CodeCoR introduces additional network communication due to its multi-agent workflow, its runtime, CPU usage, and disk overhead remain manageable.

Answer to RQ3: In summary, we evaluated the system-level cost of CodeCoR on the full HumanEval benchmark. The results show that CodeCoR has manageable overhead: it achieves competitive runtime, comparable CPU usage, and low disk write cost, while its main extra cost comes from network communication due to the multi-agent workflow.

6 How does CodeCoR perform on repository-level code generation? (RQ4)

To examine whether CodeCoR can generalize beyond standalone function-level tasks, we evaluate it on two more realistic benchmarks: DevEval [28] and RAL-Bench [35]. DevEval focuses on repository-level code generation within existing project contexts, while RAL-Bench evaluates application-level repository generation through functional system tests. We compare CodeCoR with seven representative baselines, including CoT, CodeCoT, SCoT, Self-Planning, MapCoder, CodeTree, and DebateCoder under the same backbone model and execution protocol. We report Pass@1 for DevEval and Fun. for RAL-Bench, where Fun. measures the average pass rate of functional system tests.

Table 5: Performance comparison on DevEval and RAL-Bench.

Method	DevEval	RAL-Bench	Method	DevEval	RAL-Bench
	Pass@1	Fun.		Pass@1	Fun.
CoT	4.77	16.47	CodeCoT	5.70	18.85
SCoT	2.96	16.71	Self-Planning	5.15	18.14
MapCoder	3.07	18.38	CodeTree	4.27	16.95
DebateCoder	4.11	17.90	CodeCoR	**8.88**	**20.05**

Table 5 reports the results on DevEval and RAL-Bench. CodeCoR achieves the best performance on both benchmarks, with 8.88% Pass@1 on DevEval and 20.05% Fun. on RAL-Bench. Compared with the strongest baseline, CodeCoR improves by 3.18 percentage points on DevEval and 1.20 percentage points on RAL-Bench. These results provide preliminary evidence that CodeCoR can generalize beyond standalone function-level tasks to repository-level and application-level code generation. The gains suggest that checking and pruning intermediate outputs before downstream propagation can still be useful in more realistic software development settings.

Answer to RQ4: CodeCoR achieves the best results on both DevEval and RAL-Bench, suggesting its potential for repository-level and application-level code generation.

7 Discussion

7.1 Can CodeCoR Work with Other LLMs?

This section evaluates and assesses the performance of various methods applied to two powerful LLMs: CodeLlama [40] and GPT-4 [32]. CodeLlama is a specialized model designed for coding tasks, equipped with advanced training techniques such as infilling and long context handling. Conversely, GPT-4 is a multi-modal model that excels in text comprehension and demonstrates exceptional prowess in complex reasoning tasks.

Table 6: Comparison of different methods on HumanEval and HumanEval-ET datasets

Method	GPT-4		CodeLlama (34B)	
	HumanEval	HumanEval-ET	HumanEval	HumanEval-ET
CodeChain	89.0$^{5.5\downarrow}$	61.6$^{21.9\downarrow}$	15.9$^{28.0\downarrow}$	14.0$^{23.8\downarrow}$
SCoT	78.9$^{15.6\downarrow}$	69.5$^{14.0\downarrow}$	17.4$^{26.5\downarrow}$	14.9$^{22.9\downarrow}$
Self-Planning	83.5$^{11.0\downarrow}$	76.8$^{6.7\downarrow}$	22.6$^{21.3\downarrow}$	20.1$^{17.7\downarrow}$
CodeCoT	86.6$^{7.9\downarrow}$	77.4$^{6.1\downarrow}$	34.1$^{9.8\downarrow}$	29.9$^{7.9\downarrow}$
ChatDev	84.1$^{10.4\downarrow}$	72.7$^{10.8\downarrow}$	23.6$^{20.3\downarrow}$	20.6$^{17.2\downarrow}$
MetaGPT	85.9$^{8.6\downarrow}$	74.0$^{9.5\downarrow}$	26.5$^{17.4\downarrow}$	23.1$^{14.7\downarrow}$
MapCoder	93.9$^{0.6\downarrow}$	82.9$^{0.6\downarrow}$	42.7$^{1.2\downarrow}$	37.0$^{0.8\downarrow}$
CodeCoR	**94.5**	**83.5**	**43.9**	**37.8**

Table 6 compares the Pass@1 scores of different frameworks evaluated on the CodeLlama model. For instance, in the HumanEval dataset, CodeCoR achieves a score of 43.9%, which surpasses CodeCoT's 34.1%, Self-Planning's 22.6%, SCoT's 17.4%, and CodeChain's 15.9%. Similarly, in the HumanEval-ET dataset, CodeCoR scores 37.8%, outperforming all other prompting methods.

To demonstrate the practical applicability of CodeCoR, we tested the framework on various datasets including HumanEval and HumanEval-ET using GPT-4. The results indicated significant improvements in accuracy compared to existing methods. For instance, on the HumanEval dataset, CodeCoR achieved a Pass@1 accuracy of 94.5%, and on the HumanEval-ET dataset, it achieved 83.5%. These results are significantly higher compared to other methods such as CodeChain, SCoT, Self-Planning, and CodeCoT, as shown in Table 6. We report additional results on more models and benchmarks in **our repository** for reproducibility.

7.2 How Many Tokens Are Generated by LLMs for Different Prompt Techniques?

We performed an experiment to compare the average token usage and cost per problem across different code generation methods on HumanEval and MBPP in Table 7 and Table 8. As shown in the table below, CodeCoR achieves strong performance with a reasonable cost, maintaining comparable or lower input/output token usage than other multi-agent baselines such as CodeCoT and MapCoder.

While CodeCoR incurs slightly higher cost than some single-agent methods, it achieves significantly better performance and reduces cost compared to other multi-agent frameworks, demonstrating a better balance between effectiveness and efficiency.

7.3 How does the number of repair rounds affect the performance of the agents?

In CodeCoR, the number of repair rounds is a key factor for the performance of the framework. The coding repair process of each code snippet is stopped when the code cannot be repaired to pass more test cases. In this experiment, we force the code repair process to stop by setting different numbers of repair rounds. The results in Figure 6 show that the overall performance of the four agents reaches the best when the number of repair rounds is 3.

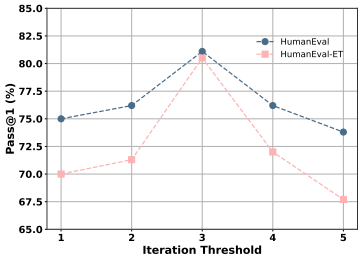

Figure 6: Pass@1 results under different repair rounds on HumanEval datasets

7.4 Hyperparameter Analysis of Candidate Budget

We analyze how the number of candidates generated by each agent affects CodeCoR on HumanEval. As shown in Table 9, cfg1–cfg4 gradually increase the numbers of prompt, test, code, and repair candidates. The prompt, code, and repair candidates are increased together to reduce the number of variables. The number of test candidates is slightly larger because generated tests are used not only for code generation, but also for scoring and pruning.

The results show that increasing the number of candidates is helpful only within a moderate range. From cfg1 to cfg3, Pass@1 increases from 87.80% to 89.63%, suggesting that more candidates can provide better choices for pruning. However, cfg4 drops to 80.49% while having the highest cost. This indicates that too many candidates may introduce more noisy intermediate outputs and make pruning harder. Therefore, CodeCoR uses cfg3 as the recommended configuration for HumanEval-like tasks.

7.5 Search Stability and Candidate Diversity Analysis

We further analyze CodeCoR from two aspects: pruning/search stability and candidate diversity. These analyses help clarify how CodeCoR's intermediate candidates behave across runs and whether multiple generated candidates provide meaningful alternatives for pruning.

1) Pruning/search stability. Table 10 shows that the retained intermediate candidates have low Jaccard overlap across three HumanEval runs. This indicates that CodeCoR may follow different intermediate search trajectories under different random seeds. However, the final pass/fail agreement remains high at 0.9350, suggesting that the final task-level outcomes are relatively stable even when the intermediate candidates vary.

2) Candidate diversity. Table 10 also shows that CodeCoR maintains non-trivial candidate diversity. Prompt diversity stays

Table 7: Comparison of Cost and Token Usage for HumanEval Dataset

Methods	Cost ($)	In-token	Out-Token
Zero-shot	0.0028	119.1	357.6
Few-shot	0.0016	374.6	58.0
CoT	0.0047	504.3	293.9
Self-planning	0.0062	521.6	364.1
SCoT	0.0058	508.8	384.2
CodeCoT	0.0071	535.5	428.7
MapCoder	0.0083	398.5	562.0
CodeCoR	**0.0064**	**517.9**	**403.2**

Table 8: Comparison of Cost and Token Usage for MBPP Dataset

Methods	Cost ($)	In-token	Out-Token
Zero-shot	0.0025	45.1	340.5
Few-shot	0.0022	578.8	157.0
CoT	0.0044	510.4	390.5
Self-planning	0.0059	578.9	488.1
SCoT	0.0052	559.7	479.6
CodeCoT	0.0050	534.0	449.1
MapCoder	0.0069	580.3	383.1
CodeCoR	**0.0047**	**521.2**	**423.6**

Table 9: Hyperparameter analysis of candidate budget on HumanEval. P/T/C/R denote prompt/test/code/repair candidates.

Config	P	T	C	R	Pass@1(%)	Cost($)
cfg1	1	1	1	1	87.80	0.0020
cfg2	2	3	2	2	88.41	0.0063
cfg3	3	5	3	3	**89.63**	0.0110
cfg4	5	7	5	5	80.49	0.0201

around 0.86, test diversity around 0.55–0.57, and code diversity around 0.47–0.49. This indicates that the generated candidates are not near-duplicates and can provide meaningful alternatives for pruning and selection.

Table 10: Search stability and candidate diversity analysis of CodeCoR on HumanEval.

Metric	Value
Avg. kept-plan Jaccard	0.0000
Avg. kept-test Jaccard	0.0577
Avg. kept-code Jaccard	0.1420
Final pass agreement	0.9350
Prompt diversity	0.8579–0.8593
Test diversity	0.5491–0.5658
Code diversity	0.4741–0.4933
Unique assertion ratio	0.6356–0.6435

Overall, CodeCoR's intermediate search process is sensitive to random seeds, but the final task-level outcomes remain relatively stable. The diversity results further show that multiple generated candidates provide useful alternatives before pruning, supporting the design of candidate generation followed by selection.

8 Threats to Validity

Our study demonstrates promising results. We have identified the following threats to validity:

We ensured a consistent experimental setup across all trials to minimize variations, although minor fluctuations in execution environments could still introduce variability. To reduce the likelihood of random factors influencing our conclusions, we conducted 10 rounds of experiments for each trial and averaged the results. Despite these efforts, LLM-based generation and pruning may still introduce stochastic variation, especially when multiple candidates are sampled and evaluated. Our ablation study validates pruning at the workflow level, but it does not fully prove that every individual pruning decision is always correct. In particular, we do not claim that every retained candidate is necessarily better than every pruned candidate. A fine-grained retained-versus-pruned candidate analysis is left for future work.

We considered construct validity by using execution-based metrics to evaluate functional correctness. Specifically, Pass@1 is used as the primary metric because it measures whether the generated code passes the benchmark-provided test suites. This choice is aligned with the main goal of code generation evaluation, namely producing functionally correct programs rather than textually similar code. CodeBLEU is reported only as a complementary metric to reflect structural and semantic similarity to reference implementations.

Although our experiments were designed to reflect practical software development scenarios, the specific datasets and settings used might still limit the broader applicability of our findings. We mitigate this limitation by extending our evaluation to full DevEval and full RAL-Bench, which cover repository-level and application-level code generation. Nevertheless, automated benchmarks cannot fully replace controlled user studies for measuring developer productivity, usability, trust, and cognitive load. To address this, we plan to validate our approach on a wider range of datasets, such as SWE-bench [20], and in different environments in future research. Developer-centered evaluation is left as future work.

9 Conclusion

This paper proposes CodeCoR, a multi-agent framework designed to realize self-reflection in code generation through intermediate output checking and pruning. It features a comprehensive workflow that allows for the checking and evaluation of intermediate outputs during the generation process. By employing four agents along with various pruning methods for pruning low-quality outputs, CodeCoR reduces the propagation of low-quality outputs and improves code generation performance, achieving an average Pass@1 score of 77.13% on four public datasets and outperforming representative prompting and multi-agent baselines under our evaluation setting. The experimental results demonstrate that the self-reflective capability of CodeCoR improves the accuracy of code generation while maintaining manageable system-level overhead. Our replication package, including source code and prompt examples, is available at https://github.com/Wwstarry/CodeCoR.

References

[1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. arXiv:2108.07732 [cs.PL] https://arxiv.org/abs/2108.07732

[2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[3] Chen et al. 2022. Codet: Code generation with generated tests. *arXiv preprint arXiv:2207.10397* (2022).

[4] Chen et al. 2023. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128* (2023).

[5] Jizheng Chen, Kounianhua Du, Xinyi Dai, Weiming Zhang, Xihuai Wang, Yasheng Wang, Ruiming Tang, Weinan Zhang, and Yong Yu. 2025. DebateCoder: Towards Collective Intelligence of LLMs via Test Case Driven LLM Debate for Code Generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 12055–12065.

[6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[8] Chowdhery et al. 2023. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research* 24, 240 (2023), 1–113.

[9] David et al. 2017. Program synthesis: challenges and opportunities. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 375, 2104 (2017), 20150403.

[10] Dong et al. 2023. Self-collaboration code generation via chatgpt. *arXiv preprint arXiv:2304.07590* (2023).

[11] Yihong Dong, Jiazheng Ding, Xue Jiang, Ge Li, Zhuo Li, and Zhi Jin. 2025. Codescore: Evaluating code generation by learning code execution. *ACM Transactions on Software Engineering and Methodology* 34, 3 (2025), 1–22.

[12] Fried et al. 2022. Incoder: A generative model for code infilling and synthesis. *arXiv preprint arXiv:2204.05999* (2022).

[13] Gulwani et al. 2017. Program synthesis. *Foundations and Trends® in Programming Languages* 4, 1-2 (2017), 1–119.

[14] Shibo Hao, Yi Gu, Haodi Ma, Joshua Jiahua Hong, Zhen Wang, Daisy Zhe Wang, and Zhiting Hu. 2023. Reasoning with language model is planning with world model. *arXiv preprint arXiv:2305.14992* (2023).

[15] Hong et al. 2023. Metagpt: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352* (2023).

[16] Huang et al. 2023. Codecot and beyond: Learning to program and test like a developer. *arXiv preprint arXiv:2308.08784* (2023).

[17] Huang et al. 2023. ScriptoriumWS: A Code Generation Assistant For Weak Supervision. In *ICLR Deep Learning for Code Workshop*.

[18] Md Ashraful Islam, Mohammed Eunus Ali, and Md Rizwan Parvez. 2024. Mapcoder: Multi-agent code generation for competitive problem solving. *arXiv preprint arXiv:2405.11403* (2024).

[19] Jiang et al. 2023. Self-planning Code Generation with Large Language Models. *ACM Transactions on Software Engineering and Methodology* (2023).

[20] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770* (2023).

[21] Le et al. 2022. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. *Advances in Neural Information Processing Systems* 35 (2022), 21314–21328.

[22] Le et al. 2023. Codechain: Towards modular code generation through chain of self-revisions with representative sub-modules. *arXiv preprint arXiv:2310.08992* (2023).

[23] Li et al. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.

[24] Li et al. 2023. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161* (2023).

[25] Li et al. 2023. Structured chain-of-thought prompting for code generation. *arXiv preprint arXiv:2305.06599* (2023).

[26] Jierui Li, Hung Le, Yingbo Zhou, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. 2025. Codetree: Agent-guided tree search for code generation with large language models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 3711–3726.

[27] Jia Li, Ge Li, Yongmin Li, and Zhi Jin. 2025. Structured chain-of-thought prompting for code generation. *ACM Transactions on Software Engineering and Methodology* 34, 2 (2025), 1–23.

[28] Jia Li, Ge Li, Yunfei Zhao, Yongmin Li, Huanyu Liu, Hao Zhu, Lecheng Wang, Kaibo Liu, Zheng Fang, Lanshen Wang, et al. 2024. Deveval: A manually-annotated code generation benchmark aligned with real-world code repositories. In *Findings of the Association for Computational Linguistics: ACL 2024*. 3603–3614.

[29] Luo et al. 2023. Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct. *arXiv preprint arXiv:2308.09583* (2023).

[30] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474* (2022).

[31] Olausson et al. 2023. Is Self-Repair a Silver Bullet for Code Generation?. In *The Twelfth International Conference on Learning Representations*.

[32] OpenAI. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL] https://arxiv.org/abs/2303.08774

[33] Ruwei Pan, Junlei Shen, Linhao Wu, Yueheng Zhu, Zixiong Yang, Yakun Zhang, Lu Zhang, and Hongyu Zhang. 2026. Toward Executable Repository-Level Code Generation via Environment Alignment. *arXiv preprint arXiv:2604.03622* (2026).

[34] Ruwei Pan, Jiangshuai Wang, Qisheng Zhang, Yueheng Zhu, Linhao Wu, Zixiong Yang, Yakun Zhang, Lu Zhang, and Hongyu Zhang. 2026. Persistent Cross-Attempt State Optimization for Repository-Level Code Generation. *arXiv preprint arXiv:2604.03632* (2026).

[35] Ruwei Pan, Yakun Zhang, Qingyuan Liang, Yueheng Zhu, Chao Liu, Lu Zhang, and Hongyu Zhang. 2026. RAL-Bench: Benchmarking for Application-Level Functional Correctness and Non-Functional Quality Attributes. *arXiv preprint arXiv:2602.03462* (2026).

[36] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. 2023. Chatdev: Communicative agents for software development. *arXiv preprint arXiv:2307.07924* (2023).

[37] Radford et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.

[38] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. Codebleu: a method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297* (2020).

[39] Giampaolo Rodola. 2020. Psutil documentation. *Psutil. https://psutil. readthedocs. io/en/latest* (2020).

[40] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).

[41] Baptiste Rozière, Jie M. Zhang, François Charton, Mark Harman, Gabriel Synnaeve, and Guillaume Lample. 2021. Leveraging Automated Unit Tests for Unsupervised Code Translation. *arXiv preprint arXiv:2110.06773* (2021). arXiv:2110.06773 [cs.PL] doi:10.48550/arXiv.2110.06773

[42] Shinn et al. 2024. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems* 36 (2024).

[43] Klaus G Troitzsch. 2009. Multi-agent systems and simulation: a survey from an application perspective. *Multi-agent systems: Simulation and applications* (2009), 53–75.

[44] Wang et al. 2023. Intervenor: Prompt the coding ability of large language models with the interactive chain of repairing. *arXiv preprint arXiv:2311.09868* (2023).

[45] Ruiqi Wang, Jiyu Guo, Cuiyun Gao, Guodong Fan, Chun Yong Chong, and Xin Xia. 2025. Can LLMs Replace Human Evaluators? An Empirical Study of LLM-as-a-Judge in Software Engineering. *Proceedings of the ACM on Software Engineering* 2, ISSTA (June 2025), 1955–1977. doi:10.1145/3728963

[46] Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi D. Q. Bui, Junnan Li, and Steven C. H. Hoi. 2023. CodeT5+: Open Code Large Language Models for Code Understanding and Generation. *arXiv preprint arXiv:2305.07922* (2023). arXiv:2305.07922 [cs.CL] doi:10.48550/arXiv.2305.07922

[47] Wei et al. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*, S. Koyejo et al. (Eds.), Vol. 35. Curran Associates, Inc., 24824–24837. https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf

[48] Welleck et al. 2022. Generating sequences by learning to self-correct. *arXiv preprint arXiv:2211.00053* (2022).

[49] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems* 37 (2024), 50528–50652.

[50] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems* 36 (2023), 11809–11822.

[51] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*. 1–12.

[52] Zhang et al. 2023. Self-edit: Fault-aware code editor for code generation. *arXiv preprint arXiv:2305.04087* (2023).
[53] Kechi Zhang, Jia Li, Ge Li, Xianjie Shi, and Zhi Jin. 2024. Codeagent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges. *arXiv preprint arXiv:2401.07339* (2024).
[54] Zheng et al. 2023. Codegeex: A pre-trained model for code generation with multilingual evaluations on humaneval-x. *arXiv preprint arXiv:2303.17568* (2023).
[55] Yueheng Zhu, Chao Liu, Xuan He, Xiaoxue Ren, Zhongxin Liu, Ruwei Pan, and Hongyu Zhang. 2025. Adacoder: An adaptive planning and multi-agent framework for function-level code generation. *IEEE Transactions on Software Engineering* (2025).